

# Problem Set 12 Design Document

**Title:** The Great Spreeland Logistics Sync  
**System:** SpreelandDispatcher Multi-Agent System  
**Course Topic:** Multi-Agent Systems and Complex Dynamics  
**Submission Artifacts:** Design PDF, Python source, A2UI JSON schema

## 1. Scenario and Objective

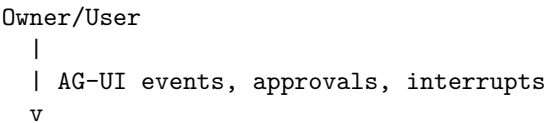
The Spreeland region faces a logistics bottleneck: local farmers need to move harvest goods to Cottbus while river levels fluctuate and bridges are partly under automated maintenance. The objective is to design a resilient Multi-Agent System that coordinates infrastructure data, expert forecasts, supplier negotiation, purchase authorization, and a live user dashboard.

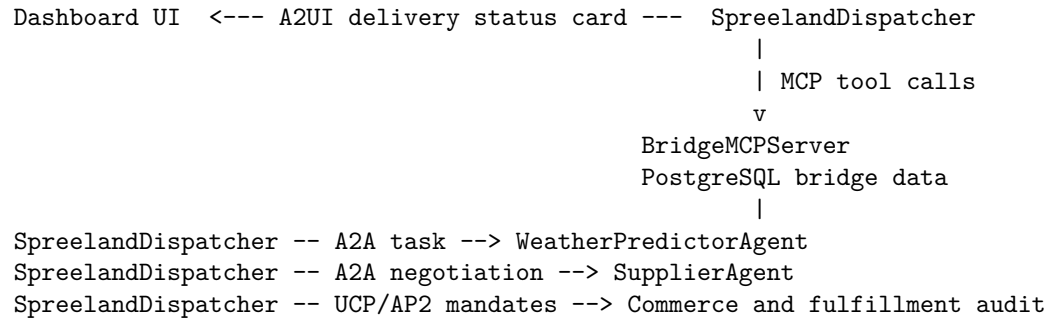
The proposed solution centers on **SpreelandDispatcher**, a coordinator agent that uses MCP for tool/data access, A2A for collaboration with external agents, UCP and AP2 for controlled commerce, and AG-UI/A2UI for user-facing streaming and dashboard rendering.

## 2. System Roles

| Role                  | Responsibility                                                                         |
|-----------------------|----------------------------------------------------------------------------------------|
| SpreelandDispatcher   | Central planner and coordinator for route, supplier, payment, and dashboard state.     |
| BridgeMCPServer       | Exposes PostgreSQL bridge status as MCP tools such as <code>get_bridge_status</code> . |
| WeatherPredictorAgent | External expert agent that returns route risk and river-level forecasts.               |
| SupplierAgent         | External agent that quotes and reserves 2 tons of Spreewald gherkins.                  |
| Owner/User            | Human or organization that authorizes purchase boundaries.                             |
| Dashboard UI          | User-facing interface receiving AG-UI events and rendering A2UI card payloads.         |

## 3. Protocol Architecture





## 4. Exercise 1: Protocol Choices

### 4.1 Infrastructure Discovery: Bridge Status from PostgreSQL

#### Protocol: MCP

The dispatcher should not connect directly to the city's PostgreSQL database. Instead, the city exposes a small MCP server with narrowly scoped tools such as `get_bridge_status(region)` and `watch_bridge_capacity(route_id)`. This matches MCP's role as a standard way for agents to connect to external tools, data sources, and workflows. The agent discovers available tools, inspects their schemas, and calls only the bridge-status operations required for the route decision.

**Why not A2A here:** the bridge database is a tool/data source, not an autonomous peer agent. A2A would add unnecessary agent-to-agent semantics.

**Submission evidence:** `src/spreeland_dispatcher.py` includes `MockBridgeMCPClient.discover_tools()` and `get_bridge_status()`.

### 4.2 Expert Consultation: Weather-Predictor Agent

#### Protocol: A2A

The weather component is owned by another team and behaves like an independent expert. It should publish an A2A Agent Card with capabilities such as `river-level-forecast` and `route-risk-assessment`. The dispatcher discovers the agent, sends a route assessment task, and receives a structured answer without depending on the weather agent's internal tools or memory.

This reflects the intended boundary between MCP and A2A: MCP equips an agent with tools, while A2A lets separate agents discover each other, delegate tasks, and exchange results.

**Submission evidence:** `discover_weather_agent()` and `request_weather_assessment()` model A2A discovery and task completion.

### 4.3 Secure Fulfillment: Authorized Purchase of 2 Tons of Gherkins

**Protocols:** UCP and AP2

The purchase path needs more than a normal API call because the agent is acting commercially on behalf of the owner. The solution uses UCP as the commerce context and policy layer: it binds the owner, budget, quantity constraints, supplier context, and approval status to the transaction. AP2 then provides the payment and audit semantics through signed mandates.

The flow is:

1. **SupplierAgent** sends an A2A offer for 2 tons of Spreewald gherkins.
2. **SpreelandDispatcher** checks UCP policy constraints such as maximum budget and maximum quantity.
3. The owner approval produces an approval token.
4. The dispatcher creates an AP2-style Intent Mandate describing the user's purchase intent and constraints.
5. The dispatcher creates an AP2-style Cart Mandate binding supplier, item, quantity, and price.
6. Fulfillment returns a receipt with a verification hash.

AP2 is appropriate because its mandate model is designed to prove user authorization, authenticity of the request, and accountability for agent-led transactions.

**Submission evidence:** `request_owner_approval()`, `create_ap2_mandates()`, and `verify_fulfillment_receipt()` implement the mocked approval, mandate, and verification flow.

### 4.4 Dynamic Visualization: Live Delivery Dashboard

**Protocols:** AG-UI and A2UI

The dispatcher streams progress as AG-UI-style events:

- `mcp.tools.discovered`
- `mcp.tool.result`
- `a2a.agent.discovered`
- `a2a.task.completed`
- `ucp.owner_approval.approved`
- `ap2.mandates.created`
- `a2ui.card.rendered`

The final dashboard component is represented as an A2UI payload. A2UI is used for the concrete delivery status card, while AG-UI is used for the live interaction channel between the agent backend and the user-facing UI.

**Submission evidence:** `schemas/delivery_status_card.json` contains the card payload, and the Python script prints the event stream with `--json`.

## 5. Exercise 2: Dispatcher Swarm Implementation

The submitted Python source is a starter implementation that can run in two modes:

- **Mock mode:** default, deterministic, no external APIs or secrets required.
- **ADK/MCP setup check:** `--show-adk-status` attempts to build a google-adk Agent with an MCP toolset, using local `.env` configuration if present.

The mock mode is intentionally the primary demonstration path because the provided `@spreeland/bridge-mcp-server` appears assignment-specific and may not exist as a public package. This keeps the submission reproducible while still showing exactly where real MCP and A2A integrations would sit.

Run:

```
cd C:\GitHub\genesis-oracle\Problem-Sets\Set12
python src\spreeland_dispatcher.py
python src\spreeland_dispatcher.py --json
python src\spreeland_dispatcher.py --write-card schemas\delivery_status_card.json
```

## 6. Security and Secret Handling

Secrets are never committed. Set12 contains its own `.gitignore` that excludes:

- `.env`
- `*.env`
- `.adk/`
- Python bytecode/cache files

If a Gemini or Google ADK key is needed, it can be copied locally from another exercise into `Problem-Sets/Set12/.env`, but the submitted code works without that key. The dispatcher also avoids hardcoding real credentials; `.env.example` documents the expected variables.

## 7. Failure Modes and Mitigations

| Failure                       | Mitigation                                                                      |
|-------------------------------|---------------------------------------------------------------------------------|
| MCP bridge server unavailable | Fall back to last known bridge snapshot or hold dispatch.                       |
| Weather agent unavailable     | Route only through bridges with conservative capacity and request human review. |
| Supplier offer exceeds budget | Reject purchase before AP2 mandates are generated.                              |
| Missing owner approval        | Stop fulfillment and emit AG-UI interrupt requesting approval.                  |

| Failure                       | Mitigation                                                 |
|-------------------------------|------------------------------------------------------------|
| Receipt verification mismatch | Mark fulfillment as disputed and do not dispatch.          |
| Route changes after dispatch  | Continue watching bridge capacity through MCP stream tool. |

## 8. Delivered Files

| File                              | Purpose                                   |
|-----------------------------------|-------------------------------------------|
| docs/Set12_Design_Document.md     | Source design document.                   |
| docs/Set12_Design_Document.pdf    | PDF export of this document.              |
| src/spreeland_dispatcher.py       | Runnable dispatcher prototype.            |
| schemas/delivery_status_card.json | A2UI delivery status card payload.        |
| .env.example                      | Local runtime configuration template.     |
| .gitignore                        | Protects local secrets and runtime state. |

## 9. Definition of Done

The assignment is complete when:

- The design document explains MCP, A2A, UCP, AP2, A2UI, and AG-UI.
- The Python source demonstrates MCP discovery and A2A negotiation.
- The purchase flow requires owner approval and creates verifiable mandates.
- The A2UI JSON is valid and contains route, bridge, weather, fulfillment, and audit fields.
- The event stream makes the dispatcher reasoning visible without exposing raw hidden chain-of-thought.

## 10. References

- Model Context Protocol documentation: <https://modelcontextprotocol.io/docs/getting-started/intro>
- A2A Protocol documentation: <https://a2a-protocol.org/latest/>
- AG-UI documentation: <https://docs.ag-ui.com/introduction>
- Google Cloud AP2 announcement: <https://cloud.google.com/blog/products/ai-machine-learning/announcing-agents-to-payments-ap2-protocol>